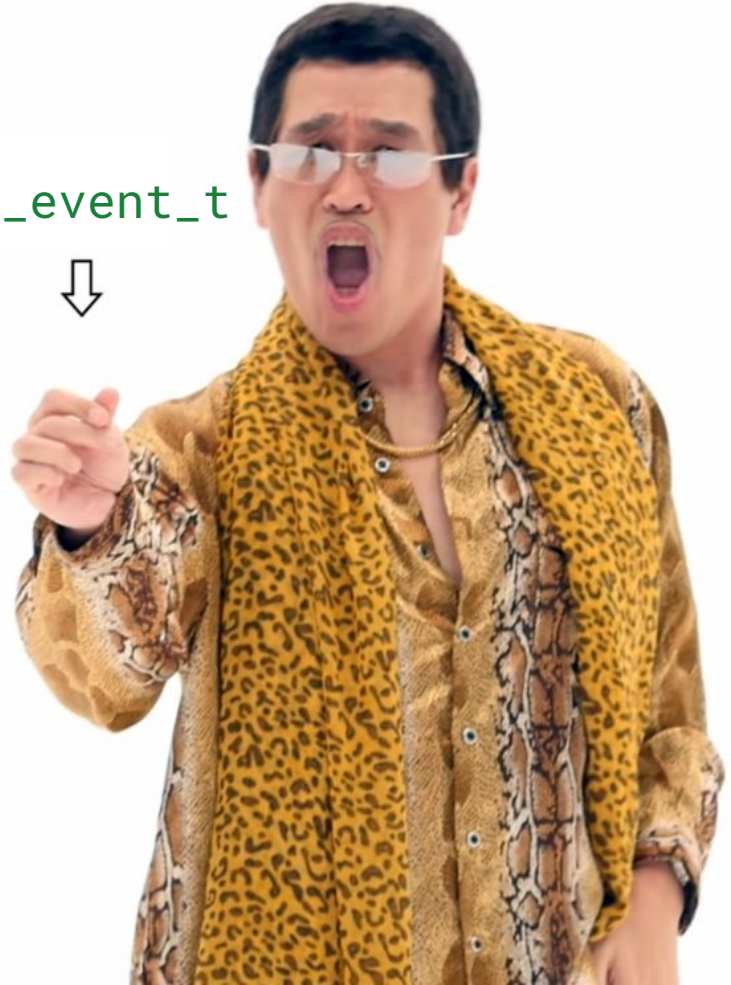
Recuperación de archivos

SistOp - Lab 3



audit_event_t

⇩



Eventos

Write
`u64 offset`
`u64 size`
`u32 num_clusters`
`u32 clusters[NMAX]`

Unlink
`u32 clusters[NMAX]`
`u32 file_size`

Mknod
`u32 dir_cluster`

Mkdir/Rmdir
`u32 dir_cluster`

Sniffing de eventos

- Información para saber los clusters ocupados por archivos en el tiempo.
- Facilidad y tamaño máximo de archivo recuperable dependiente de cómo diseñaron sus eventos.
- **Recuperación de un archivo**:
    1. Localizar evento que evidencie que el archivo fue borrado.
    2. Verificar si el archivo se puede recuperar analizando otros eventos.
    3. Si todo sale bien, reconstruir el archivo y efectivizar sus clusters.


ioctl

ioctl

Syscall de manipulación especial de archivos

```
int ioctl(int fd, unsigned long op, ... /* argp */)
```

- `fd` - algún file descriptor
- `op` - código de operación dependiente por dispositivo
- `argp` - argumento especial de usuario (`void *`)

Uso de `ioctl` en nuestro filesystem

```c
#include <fcntl.h>     // open
#include <sys/ioctl.h> // ioctl
#include "fat_types.h" // FAT_IOCTL_HELLO

int main() {
    const char *path = "/tmp/my_volume/.audit/events.bin";
    int fd = open(path, 0);
    int res = ioctl(fd, FAT_IOCTL_HELLO);
    return res;
}
```



Lo definimos arbitrariamente ahí

Hook de IOCTL en FUSE

fat_fuse_ops.c

```c
#define FAT_IOCTL_HELLO 0xF001

int fat_fuse_ioctl(/* path */, int op, /* arg, info, flags, data */) {
    if (op == FAT_IOCTL_HELLO) {
        DEBUG("ioctl: Hello world!");
    }
    return 0;
}
```

⚠️ Caso representativo

Uso de `ioctl` en nuestro filesystem

```
DEBUG audit.c:68 audit_init(): Audit file: /.audit/events.bin
DEBUG fat_fuse_ops.c:605 fat_fuse_ioctl(): ioctl: Hello world!
```

Argumentos de `ioctl`

- El número de operación está ligado a la función del 3er argumento.
- Puede ser de entrada y/o salida, o ignorado.
- El número de operación se puede codificar con macros `_IO`, `_IOR`, `_IOW`, `_IOWR`.

Argumentos de `ioctl`

- El número de operación está ligado a la función del 3er argumento.
- Puede ser de entrada y/o salida, o ignorado.
- El número de operación se puede codificar con macros `_IO,_IOR,_IOW,_IOWR`.
- Ejemplo: operación que devuelve un status sobre el estado del filesystem.

```
enum status { STATUS_HEALTHY, STATUS_CORRUPTED, ... };

#define FAT_SAMPLE     _IO('F', 0)                // ningún significado específico
#define FAT_GET_STATUS _IOR('F', 42, enum status) // obtener status del filesystem
```

Argumentos de `ioctl`

```c
int fat_fuse_ioctl(/* path */, int op, /* arg, info, flags */, void *data) {
    // op es el número de operación
    // data es el tercer argumento de ioctl
    if (op == FAT_GET_STATUS) {
        enum status *stat = data;
        *data = STATUS_HEALTHY;
    }
}
```

Estructura de un número de operación

32 31	29 ... 16	15 ... 8	7 ... 0
`dir`	`size`	`type`	`nr`
2 bits	14 bits	8 bits	8 bits

- `nr` - Número para diferenciar nuestros comandos internos.
- `type` (magic) - Identificador del driver/subsistema.
- `size` - Tamaño en bytes de la estructura del argumento (máx. 16383).
- `dir` - Dirección de la transferencia: lectura, escritura, ambas, ninguna.


Recuperación de archivos
ファイル復元

Recomendaciones

- Cuidado con los casos bordes.
  - Archivos que nunca existieron.
  - Archivos que todavía existen.
  - Archivos con nombres repetidos.
- Utilicen `DEBUG` generosamente.
- El `fd` en `ioctl` es **simbólico**, pueden NO necesitarlo estrictamente, pero es importante que al momento de usar la función, `fd` haga referencia a un archivo **válido y existente**.



Créditos: Pen Pineapple Apple Pen